

Oracle Database 11g: Administration

Support de cours - Module II

D50079EN10

Edition 1.0

June 2012

D53581



Plan du cours

- 1 **PRESENTATION GENERAL D'ORACLE**_____
- 2 **ARCHITECTURE D'UN SERVEUR ORACLE** _____
- 3 **REVISION SUR LE LANGAGE SQL** _____
- 4 **INSTALLATION ET CREATION D'UNE BASE DE DONNEES ORACLE**_____
- 5 **ADMINISTRATION D'UNE BASE DE DONNEES ORACLE**_____
- 6 **GESTION DES UTILISATEURS**_____
- 7 **GESTION DES OBJECTS** _____
- 8 **GESTION DU STOCKAGE D'UNE BASE DE DONNEES**_____
- 9 **UTILISATION DU RECOVERY MANAGER (RMAN)** _____
- 10 **SECURITE ET SURVEILLANCE DE LA BASE DE DONNEES**_____
- 11 **MINI PROJET**_____

4

INSTALLATION ET CREATION D'UNE BASE DE DONNEES ORACLE

ORACLE

1- Création d'une base de données

1.1 Pré-requis

La création de la base de données est la première étape dans l'organisation et la gestion d'un système de base de données.

La création d'une base de données permet à un ensemble de fichiers du système d'exploitation de fonctionner en tant que base de données Oracle.

Indépendamment du nombre de fichiers de données et du nombre d'instances qui accèdent à la base de données, une base de données ne peut être créée qu'une seule fois. Une base de données initiale peut être créée lors de l'installation d'Oracle. Cette base de données peut être utilisée comme base de données initiale, ou une nouvelle base de données peut être créée.

Pour créer une nouvelle base de données, il faut tenir compte de la taille des fichiers de données, des blocs, des tables et des index. L'emplacement des fichiers de contrôle doit être aussi planifié. Lors de la création d'une nouvelle base de données, il faut également tenir compte du jeu de caractères que la base de données utilisera. Une fois la base de données créée, le jeu de caractères spécifié ne peut plus être changé, sauf si la base de données est recrée.

Une base de données est créée et gérée par un DBA (Database Administrator). Si le DBA connaît les conditions préalables à la création d'une base de données, il pourra créer efficacement une base de données.

- Pour créer une base de données, un DBA doit posséder un compte privilégié authentifié par le système d'exploitation ou par un fichier de mots de passe. Un compte privilégié d'un utilisateur définit les actions que cet utilisateur peut effectuer sur la base de données ou les objets de la base de données. Lorsqu'un utilisateur d'une base de données est authentifié par le système d'exploitation, la base de données utilise les informations gérées par le système d'exploitation pour authentifier l'utilisateur.

- Il doit y avoir suffisamment de mémoire pour démarrer une instance. Ce qui signifie qu'il doit y avoir suffisamment de mémoire pour que la zone mémoire global du système (System Global Area SGA), les fichiers exécutables et les autres processus en arrière plan s'exécutent normalement.
- L'espace disque disponible sur la machine doit être suffisant pour recevoir la base de données planifiée. Le disque doit comporter suffisamment d'espace de stockage pour stocker les fichiers de contrôle, les fichiers de données et les fichiers redo log online de la base de données.

1.2 Le contenu d'une base de données après sa création

Oracle crée automatiquement plusieurs fichiers, segments et tables dans une base de données. La connaissance des éléments qui constituent une base de données permet au DBA d'identifier et de gérer les objets des bases de données.

Lors de la création de la base de données, le tablespace SYSTEM est créé.

Le tablespace SYSTEM est un ensemble de fichiers de données spécifiés dans la commande CREATE DATABASE. Il contient le dictionnaire de données.

Les tables du dictionnaire de données sont créées durant la création de la base de données. Les tables du dictionnaire de données contiennent les informations sur la structure de la base de données,

sur les utilisateurs de la base de données et sur l'allocation de l'espace. Les vues du dictionnaire de données sont créées automatiquement lors de la création de la base de données.

Les vues du dictionnaire de données sont des vues sur les tables du dictionnaire de données.

Les fichiers de contrôle et les fichiers redo log sont créés à la création de la base de données. Les fichiers de contrôle contiennent le nom de la base de données et le chemin d'accès de tous les fichiers de données et fichiers redo log. Lors de la création de la base de données, les fichiers redo log sont vides ; ils enregistrent ensuite tous les modifications apportées aux données de la base de données.

Les deux utilisateurs par défaut SYS et SYSTEM sont créés à la création de la base de données. SYS et SYSTEM ont les mots de passe par défaut « change_on_install » et « manager » respectivement sur les versions antérieures à la version 10g.

Les utilisateurs SYS et SYSTEM possèdent tous les privilèges système sur la base de données. Enfin, lorsque la base de données est créée, le segment de rollback SYSTEM est créé dans le tablespace SYSTEM. Le segment de rollback SYSTEM est utilisé pour enregistrer les modifications apportées dans le tablespace SYSTEM.

5

ADMINISTRATION D'UNE BASE DE DONNEES ORACLE

ORACLE

1- Taches de l'administrateur de Base de données

L'administrateur de base de données (appelé DBA) est responsable du contrôle et de la configuration du système et des utilisateurs.

De nombreuses taches vont être effectuées par celui-ci :

- Installer le serveur Oracle et les outils applicatifs, créer les bases de données.
- Assurer la maintenance et la disponibilité de la base de données.
- Créer la structure logique de stockage, c'est à dire les tablespaces, les tables, les vues et les indexes.
- Manager la structure physique de stockage, comprenant les fichiers de données, les fichiers de contrôle et les fichiers de Redo log.
- Allouer et prévoir l'espace disque système de stockage nécessaire aux spécifications de la base de données.
- Maintenir, Contrôler et Monitorer les accès des utilisateurs.
- Assurer la sécurité du système.
- Monitorer les performances de la base de données.
- Créer un plan de sauvegarde et de récupération

2- Le Dictionnaire de données

Les utilisateurs des bases de données, les développeurs d'applications, les administrateurs de bases de données et le serveur Oracle utilisent le dictionnaire de données comme une source centrale d'information associée à une base de données.

Le dictionnaire de données est un ensemble de tables et de vues qui est utilisé comme une référence fournissant de l'information à jour concernant la base de données.

Le dictionnaire de données est automatiquement mis à jour par Oracle lorsque des commandes de langage de définition des données ou des commandes de manipulation des données, débouchant sur l'extension d'une table, sont exécutées.

Le dictionnaire de données stocke les informations sur :

- La structure logique de la base de données

Par exemple, les informations sur les tablespaces, les blocs de données, les extents et les segments d'une base de données.

- La structure physique de la base de données

Notamment les informations sur les fichiers de contrôle, les fichiers redo log et les fichiers de données.

- Les noms et les définitions des objets

Tels que les tables, les vues, les index, les clusters, les séquences, les triggers (ou déclencheurs), les fonctions, les procédures et les packages.

Mais aussi les informations sur les espaces alloués et l'espace actuellement utilisé par les objets.

- Les contraintes d'intégrité définies pour les tables d'une base de données.
- Les noms des utilisateurs valides de la base de données et les privilèges et les rôles attribués à chaque utilisateur de la base de données.

Les privilèges définissent les opérations qu'un utilisateur peut effectuer dans la base de données.

Un rôle est constitué d'un ensemble de privilèges.

- L'audit sur une base de données.

Par exemple, le dictionnaire de données fournit les informations sur les utilisateurs qui ont accédé à des objets ou les ont mis à jour. L'audit correspond à l'enregistrement d'opérations spécifiques en fonction d'un utilisateur, d'un objet ou d'un privilège.

2.1 Composants du dictionnaire de données

Le dictionnaire de données possède deux composants :

- les tables de base
- les vues du dictionnaire de données

Les tables de base sont les tables réelles d'Oracle qui stockent les informations sur une base de données. Ces tables sont créées avec le script sql.bsq. Les informations stockées dans les tables de base sont lues et écrites par le serveur Oracle. Ces informations sont rarement accédées directement par les utilisateurs car ces informations sont normalisées et stockées sous une forme encodée.

Les utilisateurs ou administrateurs de bases de données ne doivent pas utiliser de commandes DML, telles que INSERT, UPDATE et DELETE, pour mettre à jour les tables de base directement, à l'exception de la table de trace d'audit lorsque la fonctionnalité d'audit est utilisée.

Les vues du dictionnaire de données sont des vues sur les tables de base. Elles sont créées par le script catalog.sql. Les vues du dictionnaire de données simplifient et résument les informations contenues dans les tables de base. Les vues du dictionnaire stockent également ces informations sous une forme que les utilisateurs de la base de données peuvent lire facilement.

2.2 Catégories du dictionnaire de données

Il existe trois principales catégories du dictionnaire de données :

- USER_<vues> :

Vues pouvant être accédées par les utilisateurs de la base de données. USER_<vues> se réfère au schéma d'un utilisateur de la base de données.

Un schéma est un domaine logique appartenant à un utilisateur où les objets créés par cet utilisateur peuvent être stockés.

USER_<vues> affiche les informations sur les objets appartenant à un utilisateur spécifique.

Par exemple, la vue USER_TABLES contient les informations sur les tables appartenant à un utilisateur.

USER_<vues> affiche les informations sur les privilèges et les rôles attribués par un utilisateur sur des objets créés par cet utilisateur.

- ALL_<vues>:

Vues affichant les informations sur des objets auxquels l'utilisateur peut accéder à travers l'obtention publique ou explicite de rôles et de privilèges.

- DBA_<vues>:

Vues fournissant des informations sur tous les objets de la base de données. Ces vues sont généralement interrogées par les DBA ou les utilisateurs qui possèdent le privilège système SELECT ANY TABLE. Ce privilège permet aux utilisateurs d'interroger toutes les tables de la base de données.

6

GESTION DES UTILISATEURS

1 ADMINISTRATION DES UTILISATEURS

1.1 *Création d'un utilisateur*

1.1.1 Utilisateurs; Sécurité du domaine

La sécurité est une part intégrante de l'administration de la base de données. En tant qu'administrateur, vous serez responsable de la sécurité de la base de données. Vous pourrez effectuer ceci en définissant la sécurité du domaine pour chaque utilisateur. Chaque composant de la sécurité du domaine possède des fonctions distinctes.

- Le premier composant est le verrouillage des comptes. Les comptes utilisateurs peuvent être verrouillés afin d'empêcher un ou plusieurs utilisateurs de se connecter à la base de données. Ce verrouillage pourra ce faire de manière automatique après un certain nombre d'échecs lors de l'authentification ou bien l'administrateur pourra verrouiller ou déverrouiller le compte manuellement.
- Le second composant est le tablespace par défaut. Il spécifie dans quel tablespace les objets de l'utilisateur seront stockés si celui ne spécifie par de manière explicite un tablespace lors de la création de nouveau objets. Si aucun tablespace par défaut n'est pas spécifié lors de la création de l'utilisateur, Oracle lui assignera automatiquement le tablespace SYSTEM.
- Le troisième composant est le tablespace temporaire. Celui-ci est défini par un nombre d'extents qui seront alloués par le serveur afin d'écrire des données temporaires comme des données de tri. Si ce tablespace n'est pas défini lors de la création de l'utilisateur, Oracle lui assignera automatiquement le tablespace SYSTEM.

- Le quatrième composant est les quotas pour les tablespaces. Ces quotas permettront de gérer la quantité d'espace disque alloué à un utilisateur dans les tablespaces.
- Le cinquième composant correspond aux limitations de ressources, qui permettent de contrôler l'utilisation des ressources système. Ces ressources incluent le temps CPU, le nombre de connexions pour un utilisateur.
- Le sixième composant est représenté par les privilèges système et les privilèges objets qui permettent de contrôler les accès aux données pour chaque utilisateur. Ces privilèges permettent de définir les différentes actions que les utilisateurs pourront effectuer sur les données.
- Le septième et dernier composant est représenté par les rôles qui permettent de contrôler les actions d'un utilisateur comme faisant parti d'un ensemble d'utilisateur. Les rôles peuvent être assimilés aux groupes d'utilisateurs.

1.1.2 Création d'un utilisateur : Checklist

Vous pouvez contrôler l'accès à une base de données en créant, modifiant, supprimant et en surveillant des utilisateurs. Nous allons voir les différentes étapes de la création d'un utilisateur.

La première étape consiste à choisir un nom d'utilisateur.

Lorsqu'un utilisateur est créé un schéma portant le même nom est lui aussi créé pour l'utilisateur.

Un schéma est un espace qui va contenir tous les objets de l'utilisateur comme des tables, des vues, des triggers, des procédures, des index. Un utilisateur ne peut être associé qu'à un seul schéma (portant le même nom). Quand un utilisateur se connectera il aura alors accès à tous les objets du schéma correspondant.

La seconde étape consiste à choisir les tablespaces qui serviront à stocker les objets de son schéma.

Il faudra ensuite choisir le quota d'espace disponible pour cet utilisateur sur ces différents tablespaces. Par défaut le quota est de zéro octets pour chaque tablespace.

Le quota correspond à l'espace disponible maximum pour les objets d'un utilisateur sur un tablespace. Il faut ensuite assigner un tablespace par défaut et un tablespace temporaire à ce nouvel utilisateur.

La troisième étape consiste à exécuter la commande `CREATE USER` qui va permettre de créer de manière proprement dite l'utilisateur. Il faudra ensuite lui assigner des privilèges ainsi que des rôles en fonction des différentes actions que pourra effectuer ce nouvel utilisateur.

La dernière étape consiste à former ce nouvel utilisateur pour qu'il puisse se connecter à la base de données et lui expliquer comment changer son mot de passe.

1.1.3 Création d'un utilisateur : syntaxe

Il existe deux situations distinctes pour créer un utilisateur :

- Vous créez un utilisateur quand cet utilisateur doit se connecter à une base de données et accéder à ses objets.
- Vous créez un utilisateur quand un utilisateur existant veut se connecter à une autre base de données.

Dans ce cas vous pourrez alors avoir deux ou trois comptes pour un même utilisateur.

Voici la **Syntaxe** de la commande pour créer un utilisateur.

```
CREATE USER user IDENTIFIED BY {password }  
[DEFAULT TABLESPACE default_tablespace_name]  
[TEMPORARY TABLESPACE temporary_tablespace_name]  
[QUOTA {integer [K | M] | UNLIMITED } ON tablespace_name ]  
[PASSWORD EXPIRE]  
[ACCOUNT {LOCK | UNLOCK}]  
[PROFILE {profile_name | DEFAULT}];
```

Example:

Créer l'utilisateur Pat avec le mot de pass figgg9boty avec pour tablespace par default USERS et tablespace temporaire TEMP avec un quota de 15MB et permettre a cet utilisateur de changer son password dès la premiere connexion.

```
CREATE USER pat IDENTIFIED BY figgg9boty  
DEFAULT TABLESPACE users  
TEMPORARY TABLESPACE temp  
QUOTA 15M ON users  
PASSWORD EXPIRE;
```


Remarques:

- Un utilisateur de base de données n'est pas forcément une personne. Il est courant de créer un utilisateur qui possède les objets de base de données d'une application particulière (HR, par exemple). L'utilisateur de base de données peut être une application ou simplement une entité regroupant des objets de base de données à des fins de sécurité.
- Il est préférable de choisir un mot de passe standard que l'utilisateur sera plus à même de se souvenir et de changer par la suite.
- Il peut être intéressant d'utiliser l'option EXPIRE lors de la création de l'utilisateur ce qui forcera l'utilisateur à changer son mot de passe lors de sa première connexion.
- Il est nécessaire de spécifier les tablespaces temporaire et par défaut pour tous les utilisateurs afin d'éviter que le tablespace SYSTEM soit mis par défaut et ne se fragmente trop rapidement.

1.2 Gestion des Utilisateurs

1.2.1 Gestion des mots de passe

Lors de la création d'un utilisateur le DBA assigne un mot de passe à l'utilisateur. Ce mot de passe peut être modifié par l'utilisateur ou bien par le DBA. Le DBA peut quand à lui forcer l'expiration du mot de passe.

Voici la syntaxe SQL pour changer de mot de passe:

```
ALTER USER user_name [IDENTIFIED BY {password }]  
[PASSWORD EXPIRE];
```

Le changement de mot de passe ne sera effectif que pour les sessions suivantes et non pas pour celles en cours.

1.2.2 Compte bloqués

Il peut parfois être nécessaire de bloquer temporairement un compte utilisateur. Pour rendre indisponible un compte le DBA va bloquer ce compte.

Voici la syntaxe pour bloquer ou non un compte utilisateur :

```
ALTER USER user_name  
[ACCOUNT {LOCK | UNLOCK}];
```

1.2.2 Modification du quota d'un utilisateur

Voici la syntaxe SQL pour modifier le quota d'un utilisateur sur un tablespace :

```
ALTER USER user_name  
[DEFAULT TABLESPACE default_tablespace_name]  
[TEMPORARY TABLESPACE temporary_tablespace_name]  
[QUOTA {integer [K | M] | UNLIMITED} ON tablespace_name];
```

1.2.4 Suppression d'un utilisateur

Il faut, pour pouvoir supprimer un utilisateur, que celui soit déconnecté.

Voici la syntaxe SQL pour supprimer un utilisateur :

```
DROP USER user_name [CASCADE];
```

Si l'option CASCADE est utilisée alors tous les objets de l'utilisateur seront d'abord supprimés avant de supprimer l'utilisateur. Si l'option CASCADE n'est pas utilisée alors que le schéma de l'utilisateur contient encore des objets, une erreur se produira lors de la suppression de l'utilisateur.

1.3 Monitoring des utilisateurs

1.3.1 Affichage des quotas utilisateurs

C'est grâce à la vue **DBA_TS_QUOTAS** que l'on pourra trouver toutes les informations sur les quotas des différents utilisateurs.

1.3.2 Affichage du statut des comptes utilisateurs

Vous pourrez avoir toutes les informations concernant un utilisateur grâce à la vue **DBA_USERS**. Cette vue va vous permettre d'obtenir des informations concernant les utilisateurs. Voici quelques informations intéressantes :

- USERNAME : nom de l'utilisateur.
- USER_ID : Id de l'utilisateur.
- PASSWORD : Mots de passe de l'utilisateur (encrypté pour des raisons de sécurité).
- CREATED : Date de création de l'utilisateur.
- ACCOUNT_STATUS : Statut du compte (actif ou autre...).
- LOCK_DATE : Date à laquelle le compte a été bloqué.
- EXPIRY_DATE : Date d'expiration du compte.
- DEFAULT_TABLESPACE : Tablespace par défaut du compte.
- PROFILE : Profil du compte.

Remarque :

Les Vues suivantes du dictionnaire des données permettent d'avoir des informations les utilisateurs :

ALL_USERS, USER_USERS, DBA_USERS

1.3.3 Informations contenues dans le dictionnaire de données

Le dictionnaire de données possède des vues variées qui fournissent différents types d'informations.

Vues	Description
dictionary	Vues générales
dict_columns	
dba_tables	
dba_objects	Informations sur les objets, tels que les tables, les contraintes, les gros objets et les colonnes
dba_lobs	
dba_tab_columns	
dba_constraints	
dba_users	
dba_sys_privs	Informations sur les privilèges et les rôles des utilisateurs
dba_roles	
dba_extents	
dba_free_space	Allocation d'espace pour les objets de la base de données
dba_segments	
dba_rollback_segs	Structures générales de la base de données
dba_data_files	
dba_tablespace	
dba_audit_trail	
dba_audit_objects	Informations d'audit
dba_audit_obj_opts	

La vue `DICTIONARY` du dictionnaire de données fournit une vue d'ensemble de toutes les données du dictionnaire de données. L'interrogation de la vue `DICTIONARY` évite d'avoir à se souvenir de toutes les vues et des informations fournis par chacune de ces vues.

La vue `DICTIONARY` possède deux colonnes : `TABLE_NAME` et `COMMENT`. Ces colonnes indiquent la vue qui devra ensuite être interrogée pour extraire des informations.

Par exemple, pour récupérer les informations concernant les tables accessibles à un utilisateur, il faut interroger la vue `ALL_CATALOG`.

2- GESTION DES PRIVILEGES

2.1 *Privilèges : Introduction*

2.1.1 Privilèges : Types

Les privilèges sont une part importante de la sécurité d'une base de données Oracle.

Quand un utilisateur se connecte à une base de données celui-ci est alors authentifié. Le serveur Oracle effectue cette authentification en vérifiant la validité du nom et du mot de passe de l'utilisateur.

Cette authentification constitue la première étape vers la "sécurisation" de la base de données. Les autres étapes permettant de sécuriser la base de données, est de donner des privilèges aux utilisateurs.

Il existe deux types de privilèges:

- **Les privilèges systèmes** : Ce type de privilège permet d'autoriser l'utilisateur d'effectuer un certain nombre d'opérations dans la base de donnée (comme se connecter ou bien créer modifier ou bien supprimer des objets de la base). Ces privilèges ne sont pas spécifique à un objet de schéma.
- **Les privilèges objets** : Ce type de privilège permet d'autoriser l'utilisateur à effectuer des modifications et à accéder à certains objets de la base de données comme une table, une vue. Vous pouvez donner des droits sur des objets comme UPDATE ON, DELETE ON, EXECUTE ON, SELECT ON.

Attention : Il est dangereux d'attribuer les privilèges système comportant le mot clé ANY, car ceux-ci sont très puissants et peuvent dans certains cas rendre accessible tout les objets de la base de données.

2.1.2 Privilèges système

Nous allons maintenant voir de manière plus spécifique les privilèges système.

Il faut savoir que tous les privilèges contenant le mot clé CREATE (tels que CREATE TABLE, CREATE PROCEDURE, etc...) incluent aussi les privilèges de suppression (tels que DROP TABLE, DROP PROCEDURE, etc...) car les objets de la base peuvent toujours être supprimés par leur propriétaire.

Les utilisateurs possédant le privilège CREATE TABLE ont donc aussi de manière implicite les droits de faire toutes les actions possibles sur leur table comme de créer des index (CREATE INDEX mais ce privilège n'existe pas réellement mais il existe un privilège objet INDEX) ou de lancer des analyses statistiques (ANALYSE) sur leur tables.

Les rôles sont des groupes nommés qui peuvent contenir un ou plusieurs privilèges système et être ainsi donnés à des utilisateurs ou bien à d'autres rôles.

Attention : Le privilège système UNLIMITED TABLESPACE ne peut en aucun cas être donné à un rôle. Il faut aussi le donner avec précaution car ce rôle donne accès à tout l'espace disponible dans tous les tablespaces y compris le tablespace SYSTEM.

Voici des exemples de privilèges système :

Catégorie	Exemple
TABLE	CREATE TABLE
	CREATE ANY TABLE
	SELECT ANY TABLE
	ALTER ANY TABLE
	UPDATE TABLE
	DROP ANY TABLE
SESSION	CREATE SESSION
	ALTER
	SELECT ANY TABLE
TABLESPACE	CREATE TABLESPACE
	ALTER TABLESPACE
	DROP TABLESPACE

2.1.3 Accorder des privilèges système

Voici la syntaxe qui permet de donner un privilège :

```
GRANT [{<nom du privilège>},{<nom du privilège>}]  
TO [ {<nom d'un user> | PUBLIC}]  
[WITH ADMIN OPTION];
```

Le mot clé **PUBLIC** permet de donner un privilège système ou un rôle à l'ensemble des utilisateurs.

Les mots clé **WITH ADMIN OPTION** permettent d'autoriser la personne ayant reçu ce rôle de le donner lui-même à d'autres utilisateurs ou rôles.

Example: GRANT CREATE TABLE TO emp ;

Il y a quelques points à savoir, dans l'environnement Oracle, pour donner un privilège système :

- Pour pouvoir donner un privilège système il faut être DBA ou avoir reçu explicitement ce privilège avec l'option WITH ADMIN OPTION.
- La personne ayant l'option WITH ADMIN OPTION pourra à son tour donner ce privilège système à d'autres utilisateurs avec ou sans l'option WITH ADMIN OPTION.
- L'utilisateur ayant l'option WITH ADMIN OPTION pourra donner ou retirer ce privilège système à n'importe quel utilisateur de la base de données sauf à celui lui ayant donné ce privilège.

2.3.2 Retrait de privilèges système

Il faut avoir la vision la plus précise possible des privilèges que vous allez fournir à un utilisateur car donner trop de droits à un utilisateur peut entraîner de nombreux problèmes de sécurité.

Vous devrez donc retirer des privilèges système à un utilisateur si ceux-ci ne sont plus utiles aux actions de l'utilisateur.

Voici la syntaxe :

```
REVOKE {<nom du privilège>},{<nom du privilège>}  
FROM {<nom du user> | PUBLIC} ;
```

Exemple :

```
REVOKE ALTER USER, DROP USER FROM SCOTT;
```

Remarque:

Nous avons vu précédemment qu'un utilisateur ayant reçu un privilège système avec l'option **WITH ADMIN OPTION**, pouvait à son tour donner ses droits à d'autres utilisateurs.

Il faut être extrêmement prudent avec cette option car quand vous retirerez le privilège système à cet utilisateur, tout l'autre utilisateur, ayant reçu ce même privilège de cet utilisateur, conservera ce privilège.

La révocation de ce privilège ne produit pas un effet en cascade.

2.4 Gestion des privilèges Objet

2.4.1 Accorder des privilèges objet

Les privilèges objets permettent aux utilisateurs d'effectuer des actions bien spécifiques sur des objets de la base de données, comme par exemple effacer des lignes d'une table en particulier.

Voici un tableau vous donnant une idée des privilèges objets que vous allez pouvoir utiliser sur certains objets:

Privilège Objet	Table	View	Sequence	Procédure
ALTER	X		X	
DELETE	X	X		
EXECUTE				X
INSERT	X	X		
REFERENCES	X			
SELECT	X	X	X	
UPDATE	X	X		

Il est tout à fait possible de donner ces privilèges objets à des rôles ou bien des utilisateurs.

Voici la syntaxe permettant de donner un ou plusieurs privilèges objet :

```
GRANT      {<nom du privilège> [<liste de colonne>]
ON         [,<nom du privilège> [<liste de colonne>]] ... |
TO         ALL [PRIVILEGES]}
           <nom de l'objet>
           {<nom du user> | PUBLIC}
           [{<nom du user> | PUBLIC}] ...
           [WITH GRANT OPTION] ;
```

L'option WITH GRANT OPTION permet à l'utilisateur ayant reçu ce privilège objet de donner à son tour ce privilège.

Exemple:

```
GRANT UPDATE (empno, ename)
ON emp
TO Karen
WITH GRANT OPTION;
```

Il y a certaines règles à suivre avant de donner des privilèges objets.

- Pour donner un privilège objet à quelqu'un, il faut que cet objet vous appartienne ou bien que vous ayez reçu ce privilège avec l'option WITH GRANT OPTION.
- Vous possédez tous les privilèges objets ainsi que l'option WITH GRANT OPTION pour tous les objets qui vous appartiennent.
- Pour des raisons de sécurité, faites très attention lorsque vous donnez des droits sur vos objets.
- L'option WITH GRANT OPTION ne peut être utilisée lorsque vous donnez un privilège objet à un rôle.

2.4.2 Retrait de privilèges objet

Il faut toujours se rappeler que donner trop de droits peut engendrer des problèmes de sécurité.

Nous allons donc voir comment retirer des droits à des utilisateurs ou à des rôles.

Voici la syntaxe :

```
REVOKE {<nom du privilège> [, <nom du privilège>]  
ON      <nom de l'objet>  
FROM [ {<nom du user> | PUBLIC }]  
      [CASCADE CONSTRAINTS];
```

A la différence de l'option WITH ADMIN OPTION, l'option WITH GRANT OPTION se retire en cascade. Si l'utilisateur à qui vous aviez donné un privilège avec l'option WITH GRANT OPTION l'avait donné à son tour à un autre utilisateur, et bien lorsque vous retirez ce privilège à cet utilisateur, alors tous ceux qui ont reçu ce même privilège de cet utilisateur se le verront retirer automatiquement.

- **Affichage des privilèges système**

Vous pourrez en tant que DBA aller chercher toutes les informations concernant les privilèges système dans les vues système DBA_SYS_PRIVS, et SESSION_PRIVS.

- **Affichage des privilèges objet**

Vous pourrez trouver les informations concernant les privilèges objets dans les vues DBA_TAB_PRIVS et DBA_COL_PRIVS.

2.5 Gestion des Rôles

2.5.1 Caractéristiques des rôles

Les rôles ont été mis en place afin de faciliter l'administration des privilèges de la base de données.

Ces rôles :

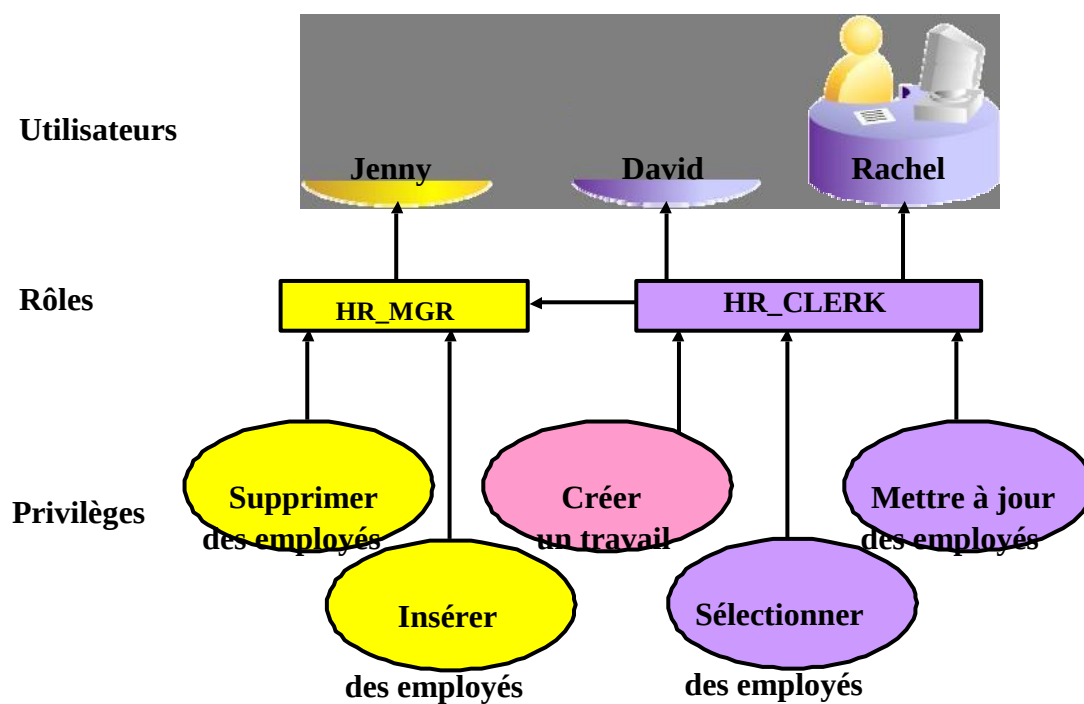
- Peuvent être composé de privilèges système et objet.
- Peuvent être donné à des utilisateurs et à des rôles (excepté lui-même).
- Les rôles peuvent être activés ou désactivés.
- Le nom d'un rôle ne pourra en aucun cas être celui d'un autre rôle ou celui d'un utilisateur.
- Les rôles n'appartiennent à aucun utilisateur et ne font partis d'aucun schéma.
- La description et les privilèges associés à un rôle sont stockés dans le dictionnaire de données.
- Pour donner ou retirer un droit il suffit d'utiliser la même commande que pour donner un privilège.

5.1.2 Bénéfices des rôles

L'utilisation des rôles permet de :

- Réduire le besoin de donner des droits de manière individuelle. Grâce aux rôles vous pouvez donner des droits à un rôle et donner ce rôle à des utilisateurs, ce qui vous évitera de donner tous ces droits à chaque utilisateur.
- Gérer de manière dynamique la gestion des privilèges. Si vous modifiez les privilèges associés à un rôle, alors tout les utilisateurs, possédant ce rôle, obtiendront les modifications de manière transparente et automatique.
- Gérer la sécurité en activant ou désactivant ces rôle, et en les sécurisant au moyen d'un mot de passe.
- Supprimer les effets de suppressions en cascade absent lors de la suppression de privilèges système ayant été fournis avec l'option WITH ADMIN OPTION.
- Diminuer le stockage de tous les ordres GRANT dans le dictionnaire de données.
- Réduire le nombre de privilèges à vérifier si le rôle est désactivé. Si le rôle est désactivé il faudra moins de temps pour vérifier les droits de l'utilisateur lors de l'exécution d'une requête.

Affecter des privilèges à des rôles et des rôles à des utilisateurs



2.5.2 Création des Rôles

Voici la syntaxe qui permet de créer un rôle :

```
CREATE ROLE <nom du rôle>  
[NOT IDENTIFIED | IDENTIFIED {BY <mot de passe> };
```

5.2.2 Identification des rôles prédéfinis

CONNECT	Permet à l'utilisateur de se connecter et de crée des objets à la condition que vous lui donniez un quota sur un tablespace.
DBA	Donne tout les privilèges système avec l'option WITH ADMIN OPTION
EXP_FULL_DATABASE	Permet de faire des exports complets de la base de données.
IMP_FULL_DATABASE	Permet de faire des imports complets de la base de données.
DELETE_CATALOG_ROLE	Permet de vider les tables d'audit.
EXECUTE_CATALOG_ROLE	Permet d'exécuter les packages PL/SQL système.
SELECT_CATALOG_ROLE	Permet d'avoir accès sur les tables et vues du dictionnaire de données.

Il existe plusieurs rôles qui sont définis automatiquement pour les bases de données Oracle lorsque vous exécutez les scripts de création de la base. Le rôle `CONNECT` est accordé automatiquement à tout utilisateur créé avec Enterprise Manager.

5.2.3 Modifier un rôle

La seule chose qu'il est possible de modifier dans un rôle est son système de protection par mots de passe. Ceci peut être fait grâce à la syntaxe suivante:

```
ALTER ROLE <nom du rôle>
{ NOT IDENTIFIED | IDENTIFIED { BY <mot de passe> } };
```

5.2.4 Assignment d'un rôle à un utilisateur

Voici la syntaxe pour assigner un rôle à un utilisateur ou à un autre rôle :

```
GRANT <nom du rôle> [, <nom du rôle>] ...
TO { <nom du user> | <nom du rôle> | PUBLIC }
[, { <nom du user> | <nom du rôle> | PUBLIC } ] ...
[WITH ADMIN OPTION];
```

5.3.2 Activation d'un rôle

Il est possible d'activer ou de désactiver un rôle temporairement pour une session. Quand un rôle est activé l'utilisateur pourra alors utiliser les privilèges de ce rôle.

Voici la syntaxe qui permet d'activer ou de désactiver un rôle :

```
SET ROLE { <nom du rôle> [IDENTIFIED BY PASSWORD]
[, <nom du rôle> [IDENTIFIED BY PASSWORD]] ... | ALL
[EXCEPT <nom du rôle> [, <nom du rôle>] ...] | NONE};
```

Cette commande désactive tous les rôles et réactive seulement ceux cités dans la commande.

5.3.3 Retrait d'un rôle à un utilisateur

Voici la syntaxe qui permet de retirer un rôle à un utilisateur ou à un autre rôle :

```
REVOKE <nom du rôle> [, <nom du rôle>] ...
FROM { <nom du user> | <nom du rôle> | PUBLIC }
[, { <nom du user> | <nom du rôle> | PUBLIC } ] ...;
```

5.3.4 Suppression d'un rôle

Afin de s'assurer qu'un rôle ne peut plus être utilisé, il vaut mieux supprimer le rôle avec la syntaxe suivante :

```
DROP ROLE <nom du rôle>;
```

5.4.1 Conseils pour la création de rôles

Nous allons voir les différents points qu'il faut suivre pour mettre en place des rôles appropriés aux besoins des utilisateurs.

- Créer autant de rôle que de fonctions existantes.
- Assigner les privilèges nécessaires pour effectuer une fonction au rôle correspondant.
- Créer un rôle (rôle utilisateur) distinct pour chaque type d'utilisateur.
- Donner au rôle utilisateur les rôles des différentes fonctions que ce type d'utilisateur pourra effectuer.
- Donner à chaque utilisateur son rôle utilisateur.
- Donner un mot de passe à chaque rôle.

5.4.2 Affichage des infos concernant les rôles

Vous pourrez trouver des informations sur les différents rôles dans les vues système suivantes :

- `ROLE_SYS_PRIVS` : vue concernant les privilèges système accordés à des rôles.
- `ROLE_TAB_PRIVS` : vue concernant les privilèges sur une table accordés à des rôles.
- `ROLE_ROLE_PRIVS` : vue concernant les rôles accorder à d'autres rôles.
- `DBA_SYS_PRIVS` : vue permettant de voir les privilèges système accordés à des utilisateurs et à des rôles.
- `DBA_ROLES` : vue permettant d'accéder à tous les rôles présents dans la base de données.
- `DBA_ROLE_PRIVS` : vue permettant les rôles qui ont été accordés à d'autres rôles ou utilisateurs.

EXERCICE

1. Créer un utilisateur oracle ayant votre nom **XXXx** avec les caractéristiques suivantes :
 - tablespace par défaut : users
 - tablespace temporaire : temp
 - mot de passe : à votre choix mais il doit être changé dès la première connexion.
2. Connectez vous comme étant XXXx. Que constatez-vous ?
3. Etant System, Affectez le privilège CREATE SESSION à l'utilisateur et reconnectez vous comme étant XXXx. Si la connexion a réussi, tentez de créer la table **PILOTE (PLNUM, PLNOM, PLPRENOM, VILLE, SALAIRE)** en s'assurant que le salaire de base des pilotes est supérieur à 6000000. Que constatez-vous?
Remedier au problème et inserer cinq (5) enregistrements dans la table pilote.
4. Créer un autre utilisateur nommé **bert** avec comme mot de pass bert. Le tablespace temporaire de bert devra être TEMP, et il faudra s'assurer que bert ne pourra créer aucun n'objets dans tablespace par SYSTEM. Donner à bert un quota illimité sur le tablespace users
5. Examiner la vue DBA_USERS et DBA_TS_QUOTAS pour voir les informations relatives aux users bert et XXXx
6. Interroger la vue qui affichera tous les utilisateurs
7. Créer à partir du compte de l'utilisateur bert les tables emp et dept localisées dans le schema de l'utilisateur scott.
8. Retiré le quota illimité de bert sur le tablespace users et lui donner a la place un quota de 50 MB sur le tablespace users.

9. Etant System, retirer les privilèges accordés à XXX et supprimez la table qu'il à créée

10. Etant System, créer un rôle RL_XXX qui permettent à l'utilisateur XXX de se connecter, de créer des tables, d'insérer des lignes dans ses tables et de créer des clusters

11. Créer un rôle RL_ADMINx qui vous permet de supprimer, consulter et modifier des tables dans n'importe quel schéma. Ce rôle doit être créé avec un mot de passe.

12. Affectez les roles RL_XXX et RL_ADMINx à l'utilisateur XXX.

13. Connectez-vous comme étant XXX.

Créer la table à AVION (AVNUM, AVNOM, CAPACITE, LOCALISATION).

Inserer dans la tables AVION trois (3) enregistrements.

14. Etant System, tentez de donner à bert, le droit de consulter, insérer et modifier les tables de XXX. Que constatez vous ? Faire la même chose étant XXX et refaire le test avec l'utilisateur bert.



THESE eKIT MATERIALS ARE FOR YOUR USE IN THIS CLASSROOM ONLY